```
// Free Studio (mobile): serves
the phone app and handles
generation requests.
// Requires Node.js 18 or newer.
No installs needed.
//
// Environment variables:
//   PORT           the port to
listen on (hosts usually set this
for you)
//   VIDEO_API_URL  the address
your video provider gives you
//   VIDEO_API_KEY  your key from
that provider (never sent to the
phone)

const http = require("http");
const fs = require("fs");
const path = require("path");

const PUBLIC_DIR = __dirname;
const PORT =
Number(process.env.PORT) || 8000;
const VIDEO_API_URL =
```

```
(process.env.VIDEO_API_URL ||
"").trim();
const VIDEO_API_KEY =
(process.env.VIDEO_API_KEY ||
"").trim();
const MAX_BODY = 64 * 1024;

const STYLE_TEXT = {
  photo: "photographic, natural
light, sharp focus",
  painting: "oil painting, visible
brushwork, warm palette",
  pixel: "pixel art, limited
palette, 16-bit",
  ink: "black ink sketch, loose
linework, paper texture",
};

function send(res, status, body,
headers = {}) {
  res.writeHead(status, { "Cache-
Control": "no-store", ...headers
});
  res.end(body);
}

function sendJson(res, status,
```

```
obj) {
  send(res, status,
JSON.stringify(obj), { "Content-
Type": "application/json" });
}

function readJson(req) {
  return new Promise((resolve,
reject) => {
    let size = 0;
    const chunks = [];
    req.on("data", (c) => {
      size += c.length;
      if (size > MAX_BODY) {
        reject(new
Error("too_large"));
        req.destroy();
      } else chunks.push(c);
    });
    req.on("end", () => {
      try {

resolve(JSON.parse(Buffer.concat(c
hunks).toString("utf8") || "{}"));
      } catch {
        reject(new
Error("bad_json"));
```

```
      }
    });
    req.on("error", reject);
  });
}

function slugify(text) {
  return (text.replace(/[^a-zA-Z0-
9]+/g, "-").replace(/^-|-$/g,
"").toLowerCase().slice(0, 40)) ||
"creation";
}

async function makeImage(prompt,
styles) {
  const extras = (styles ||
[]).filter((s) =>
STYLE_TEXT[s]).map((s) =>
STYLE_TEXT[s]);
  const fullPrompt = [prompt,
...extras].join(", ");
  const url =

"https://image.pollinations.ai/pro
mpt/" +
    encodeURIComponent(fullPrompt)
+
```

```
    "?
width=768&height=768&nologo=true&s
eed=" +
    Math.floor(Math.random() *
1e6);
  const res = await fetch(url, {
signal:
AbortSignal.timeout(180000) });
  if (!res.ok) throw new
Error("upstream_" + res.status);
  const type =
res.headers.get("content-type") ||
"image/jpeg";
  const data = Buffer.from(await
res.arrayBuffer());
  return { data, type, ext:
type.includes("png") ? "png" :
"jpg" };
}

async function makeVideo(prompt) {
  if (!VIDEO_API_URL) throw new
Error("no_video_provider");
  const headers = { "Content-
Type": "application/json" };
  if (VIDEO_API_KEY)
headers.Authorization = "Bearer "
```

```
+ VIDEO_API_KEY;
  const res = await
fetch(VIDEO_API_URL, {
    method: "POST",
    headers,
    body: JSON.stringify({ prompt
}),
    signal:
AbortSignal.timeout(900000),
  });
  if (!res.ok) throw new
Error("upstream_" + res.status);
  const result = await res.json();
  if (!result.url) throw new
Error("no_video_url");
  const videoRes = await
fetch(result.url, { signal:
AbortSignal.timeout(900000) });
  if (!videoRes.ok) throw new
Error("upstream_video_" +
videoRes.status);
  return { data: Buffer.from(await
videoRes.arrayBuffer()), type:
"video/mp4", ext: "mp4" };
}

const ERRORS = {
```

```
  no_video_provider: [503, "Video
isn't connected yet. The site
owner needs to add a video
provider."],
  no_video_url: [502, "The video
service didn't return a video. Try
a different prompt."],
  bad_json: [400, "Something about
the request was wrong. Try
again."],
  too_large: [413, "That request
was too long."],
};

async function handleApi(req, res,
pathname) {
  let body;
  try {
    body = await readJson(req);
  } catch (err) {
    const [status, message] =
ERRORS[err.message] || [400,
"Something went wrong. Try
again."];
    return sendJson(res, status, {
error: message });
  }
```

```
  const prompt =
String(body.prompt ||
"").trim().slice(0, 1000);
  if (!prompt) return
sendJson(res, 400, { error:
"Describe what to make first." });

  try {
    let out;
    if (pathname === "/api/image")
out = await makeImage(prompt,
body.styles);
    else if (pathname ===
"/api/video") out = await
makeVideo(prompt);
    else return sendJson(res, 404,
{ error: "Not found." });

    const name =
`${slugify(prompt)}.${out.ext}`;
    send(res, 200, out.data, {
      "Content-Type": out.type,
      "Content-Disposition":
`attachment; filename="${name}"`,
      "Access-Control-Expose-
Headers": "Content-Disposition",
```

```
    });
  } catch (err) {
    if (ERRORS[err.message]) {
      const [status, message] =
ERRORS[err.message];
      return sendJson(res, status,
{ error: message });
    }
    const timedOut = err.name ===
"TimeoutError" || err.name ===
"AbortError";
    sendJson(res, 502, {
      error: timedOut
        ? "That took too long. Try
a shorter description."
        : "The generation service
is busy or unavailable. Try again
in a moment.",
    });
  }
}

const TYPES = {
  ".html": "text/html;
charset=utf-8",
  ".webmanifest":
"application/manifest+json",
```

```
};

const server =
http.createServer((req, res) => {
  const { pathname } = new
URL(req.url, "http://localhost");

  if (req.method === "POST" &&
pathname.startsWith("/api/")) {
    return handleApi(req, res,
pathname);
  }

  if (req.method === "GET") {
    const file = pathname === "/"
? "index.html" :
pathname.replace(/^\//, "");
    const full =
path.normalize(path.join(PUBLIC_DI
R, file));
    if
(!full.startsWith(PUBLIC_DIR))
return send(res, 403,
"Forbidden");
    fs.readFile(full, (err, data)
=> {
      if (err) return send(res,
```

```
404, "Not found");
      send(res, 200, data, {
"Content-Type":
TYPES[path.extname(full)] ||
"application/octet-stream" });
    });
    return;
  }

  send(res, 404, "Not found");
});

server.listen(PORT, () => {
  console.log(`Free Studio is
running on port ${PORT}`);
  if (!VIDEO_API_URL)
console.log("Video is not
connected yet. Images work now.");
});
```